

WEEK-3(OS)

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/wait.h>

int main() {
    pid_t pid = fork();

    if (pid < 0) {
        perror("Fork failed");
        return 1;
    }

    if (pid == 0) {
        // --- CHILD PROCESS ---
        printf("[Child] PID: %d. Transitioning to WAITING (Sleeping)...\n", getpid());
        sleep(10); // Forces Interruptible Sleep State (S)

        printf("[Child] Transitioning to RUNNING...\n");
        // Infinite loop to force Running State (R)
        unsigned long long i = 0;
        while(i < 5000000000ULL) {
            i++;
        }

        printf("[Child] Exiting. Transitioning to TERMINATED...\n");
        exit(0);
    } else {
        // --- PARENT PROCESS ---
        printf("[Parent] PID: %d. Waiting before reaped...\n", getpid());
        sleep(25); // Parent sleeps long enough to let child become a Zombie (Z)

        wait(NULL); // Reaps the child
        printf("[Parent] Child reaped. Exiting.\n");
    }
    return 0;
}
```

output

```
ashwika@ashwikapadamata:~$ gcc weekthree.c -o weekthree
ashwika@ashwikapadamata:~$ ./weekthree
[Parent] PID: 735. Waiting before reaped...
[Child] PID: 736. Transitioning to WAITING (Sleeping)...
[Child] Transitioning to RUNNING...
[Child] Exiting. Transitioning to TERMINATED...
[Parent] Child reaped. Exiting
ashwika@ashwikapadamata:~$
```

week2(os)

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/wait.h>
#include <string.h>

int main() {
    char command[100];

    // 1. Accept a Linux command as input from the user
    printf("Enter a Linux command to execute: ");
    if (fgets(command, sizeof(command), stdin) == NULL) {
        perror("Error reading input");
        return 1;
    }

    // Remove the trailing newline character from fgets
    command[strcspn(command, "\n")] = '\0';

    // 2. Create a child process using fork()
    pid_t pid = fork();

    if (pid < 0) {
        // Fork failed
        perror("Fork failed");
        return 1;
    }
    else if (pid == 0) {
        // --- CHILD PROCESS ---
        // 5. Display the Process ID (PID) of the child process
        printf("\n[Child] Child Process PID: %d\n", getpid());
        printf("[Child] Executing command: %s\n\n", command);

        // 3. Execute the command using an appropriate exec() system call
        // execlp searches the PATH environment variable for the command
        execlp(command, command, (char *)NULL);

        // If execlp returns, an error occurred
        perror("Execution failed");
        exit(1);
    }
    else {
        // --- PARENT PROCESS ---
        // 5. Display the Process ID (PID) of the parent process
        printf("[Parent] Parent Process PID: %d\n", getpid());
    }
}
```

```
printf(" [Parent] waiting for the child process to complete\n");

// 4. Allow the parent process to wait for the child using wait()
int status;
wait(&status);

printf("\n[Parent] Child process finished execution.\n");

return 0;
```

output

```
ashwika@ashwikapadamata:~$ ./week
Enter a Linux command to execute: pwd
[Parent] Parent Process PID: 670
[Parent] Waiting for the child process to complete...

[Child] Child Process PID: 673
[Child] Executing command: pwd

/home/ashwika

[Parent] Child process finished execution.
ashwika@ashwikapadamata:~$ █
```